Pandas

📘 Pandas Stages 1–6 — Complete Cheat Sheet & Study Notes

A structured guide covering all fundamental and practical pandas operations:

Core concepts → CSV CRUD → Selection/Indexing → Cleaning → Grouping/Aggregation → Visualization

---

🧩 Stage Overview

Stage	Topic	Focus
1	Core Concepts	Series, DataFrame, Index
2	CSV CRUD	Create / Read / Update / Delete / Safe Save
3	Selection & Indexing	loc / iloc / query / conditional filters
4	Cleaning	Missing values, duplicates, string ops, type conversion
5	Grouping & Aggregation	groupby / agg / transform / pivot
6	Visualization	plot / hist / boxplot / scatter / bar

0️⃣ Imports & Setup

```
# Import core libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Inline plotting for Jupyter or Google Colab
%matplotlib inline
```

1️⃣ Core Concepts & Data Structures

```
# --- 1.1 Create Series (1D labeled array)
s = pd.Series([10, 20, 30], index=['a','b','c'])

# --- 1.2 Create DataFrame (2D labeled table)
df = pd.DataFrame({'name':['Tom','Jerry'], 'age':[20,21]}, index=['s1','s2'])

# --- 1.3 Inspect structure and metadata
print(df.shape)       # number of rows × columns
print(df.columns)     # column labels
print(df.index)       # row index
print(df.dtypes)      # data types per column
df.info()             # concise summary
df.describe()         # statistical summary (for numeric columns)
df.head(3)            # preview top 3 rows
df.tail(3)            # preview last 3 rows

# --- 1.4 Change index or transpose
df = df.set_index('name')       # set column as index
df = df.reset_index()           # reset index to default
df.T                            # transpose rows ↔ columns
```

🧠 **Concepts**

- Series: 1D labeled array
- DataFrame: 2D table (rows + columns)
- Index = row labels; Columns = field names

2️⃣ CSV I/O — Create / Read / Update / Delete / Safe Save

```
# --- 2.1 Write CSV (export)
df.to_csv(
    'students.csv',
    index=False,         # don't save row index
```

```
    encoding='utf-8-sig', # make Excel-compatible
    sep=',',            # separator
    na_rep=''           # fill NaN with empty string
)

# --- 2.2 Read CSV (import)
df = pd.read_csv(
    'students.csv',
    sep=',',
    header=0,                 # first line = header
    na_values=['NA','?',''],     # treat these as NaN
    encoding='utf-8-sig'
)

# --- 2.3 Update values
df['total'] = df['math'] + df['eng']             # create new column
df.loc[df['age'] < 18, 'group'] = 'minor'         # conditional update

# --- 2.4 Append new rows
new_row = pd.DataFrame([{'math':90,'eng':85}], index=['new'])
df = pd.concat([df, new_row])                    # add new record

# --- 2.5 Delete rows/columns
df = df.drop(columns=['col1'], errors='ignore')    # drop column safely
df = df.drop(index=df[df['score'] < 60].index)     # delete by condition

# --- 2.6 Safe saving using tempfile (atomic write)
import tempfile, os
tmp = tempfile.NamedTemporaryFile('w', delete=False, newline='', encoding
='utf-8-sig')
df.to_csv(tmp.name, index=False, encoding='utf-8-sig')
os.replace(tmp.name, 'students_safe.csv')
```

💡 Tips

- Always use utf-8-sig to avoid garbled text in Excel

- Use tempfile + os.replace for **atomic safe save**

3️⃣ Selection & Indexing

```
# --- 3.1 Column access
df['math']                  # single column
df[['math','eng']]          # multiple columns

# --- 3.2 Row access
df.loc['ann']               # by label
df.iloc[0:3, 1:4]           # by numeric position (end exclusive)

# --- 3.3 Conditional filtering
df.loc[df['math'] > 85]       # all rows where math > 85
df.loc[(df['math'] > 85) & (df['eng'] > 90), ['math','eng']]
df.query("math > 85 and eng > 90")  # SQL-like syntax

# --- 3.4 Single-cell access
df.at['ann','math']         # by label
df.iat[0, 1]                # by numeric position
```

🧠 **Concepts**

- .loc[] → label-based, inclusive end
- .iloc[] → position-based, exclusive end
- query() → readable filtering like SQL WHERE clause

4️⃣ Cleaning — Missing, Duplicates, Text, Types

```
# --- 4.1 Check missing data
df.isnull().sum()           # count nulls per column

# --- 4.2 Handle missing data
df = df.dropna(subset=['math'])              # drop rows with missing math
```

```
df['cs'] = df['cs'].fillna(df['cs'].mean())      # fill NA with mean

# --- 4.3 Replace placeholder values
df = df.replace(['N/A',''], pd.NA)              # convert placeholders to NaN

# --- 4.4 String operations
df['class'] = df['class'].astype(str).str.strip().str.upper()
# remove spaces and unify case

# --- 4.5 Data type conversion
df['score'] = pd.to_numeric(df['score'], errors='coerce')   # safely convert to
numeric
df['date'] = pd.to_datetime(df['date'], errors='coerce')    # convert to datetime
df['id'] = df['id'].astype('Int64')                         # use nullable integer type

# --- 4.6 Remove duplicates
df = df.drop_duplicates(subset=['name'], keep='first')
```

💡 **Tips**

- Use errors='coerce' to safely convert invalid entries → NaN
- Always clean strings (.str.strip()) before grouping or comparing
- Pandas Int64 supports missing values (unlike plain int64)

5️⃣ Grouping & Aggregation

```
# --- 5.1 Simple group stats
df.groupby('class').mean()                      # average per class
df.groupby('class')[['math','eng']].agg(['mean','max','min'])

# --- 5.2 Custom aggregation
def score_range(x): return x.max() - x.min()    # custom function
summary = df.groupby('class').agg(
    math_mean=('math','mean'),
    math_range=('math',score_range),
```

```
    count=('math','count')
)

# --- 5.3 Transform (apply per-group result back)
df['math_z'] = df.groupby('class')['math'].transform(
    lambda x:(x-x.mean())/x.std()
)

# --- 5.4 Pivot table (Excel-like)
pv = pd.pivot_table(df, values='math', index='class', aggfunc='mean')
```

🧠 **Concepts**

- groupby() = SQL “GROUP BY” equivalent
- agg() supports dicts, lists, or custom functions
- transform() applies per-group operation **and returns full-length column**

6️⃣ Visualization (pandas + matplotlib)

```
# --- 6.1 Line chart (multi-column)
df[['math','eng','cs']].plot(kind='line', title='Scores by Subject')

# --- 6.2 Bar chart with average line
df['avg'].plot(kind='bar', title='Average per Student')
plt.axhline(df['avg'].mean(), color='orange', ls='--', label='Mean')
plt.legend()

# --- 6.3 Histogram
df['avg'].plot(kind='hist', bins=6, edgecolor='black', title='Avg Distribution')

# --- 6.4 Boxplot (distribution + outliers)
df[['math','eng','cs']].plot(kind='box', title='Subject Distributions')
```

```
# --- 6.5 Scatter plot (relationship between 2 vars)
df.plot(kind='scatter', x='math', y='cs', title='Math vs CS')
```

💡 **Tips**

- .plot(kind='bar') → categorical comparison
- .plot(kind='hist') → distribution
- .plot(kind='scatter') → correlation check
- .plot(kind='box') → show median, quartiles, and outliers

⚙️ Common Pitfalls & Pro Tips

✅ Avoid chained indexing like df[df.x>0]['y']=...
→ Use df.loc[df.x>0, 'y']=...

✅ Prefer assignment style (df = df.drop(...)) over inplace=True

✅ loc uses labels (slice end inclusive)
iloc uses numeric positions (slice end exclusive)

✅ Use encoding='utf-8-sig' for Excel-friendly CSVs

✅ Always check dtypes after reading files

✅ Safe writing: tempfile + os.replace prevents partial save

🚀 Minimal Daily Template

```
import pandas as pd, numpy as np, tempfile, os

# --- 1) Load file
df = pd.read_csv('input.csv', encoding='utf-8-sig')
```

```
# --- 2) Clean
df = df.replace(['NA','N/A',''], pd.NA)
df.columns = df.columns.str.strip().str.lower()
for c in ['math','eng','cs']:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors='coerce')
if 'class' in df.columns:
    df['class'] = df['class'].astype(str).str.strip().str.upper()
df = df.dropna(subset=[c for c in ['math','eng','cs'] if c in df.columns])

# --- 3) Analyze
if all(c in df.columns for c in ['math','eng','cs']):
    df['avg'] = df[['math','eng','cs']].mean(axis=1)
    g = df.groupby('class')[['math','eng','cs','avg']].agg(['mean','max','min','coun
t']).round(2)
    g.columns = ['_'.join(map(str, c)) for c in g.columns]
else:
    g = df.describe(include='all')

# --- 4) Save safely
tmp = tempfile.NamedTemporaryFile('w', delete=False, newline='', encoding
='utf-8-sig')
g.to_csv(tmp.name, encoding='utf-8-sig')
os.replace(tmp.name, 'summary.csv')
```